

Relatório Técnico

Agente de IA para Triagem de Logs de Segurança

cenourin

Portfólio técnico — IA aplicada a Cibersegurança

15 de julho de 2026

Abstract

Este documento descreve, de forma consolidada, os requisitos, a arquitetura, o modelo de dados, o plano e a execução de testes, a análise dos resultados obtidos e uma proposta de aplicação em cenário real do *Agente de IA para Triagem de Logs de Segurança*. O sistema classifica eventos de log (SSH, firewall, EDR) em **SUSPEITO**, **NORMAL** ou **CRITICO**, gera um resumo em linguagem natural e sugere uma ação de resposta, utilizando um modelo de linguagem (LLM) quando disponível ou um classificador heurístico determinístico como alternativa local.

Contents

1	Introdução e contexto	1
2	Documento de requisitos	1
2.1	Escopo	1
2.2	Atores	2
2.3	Requisitos funcionais	2
2.4	Requisitos não funcionais	2
2.5	Casos de uso	3
2.6	Critérios de aceite	3
3	Arquitetura e engenharia de software	3
3.1	Visão geral	3
3.2	Módulos	3
3.3	Decisão de arquitetura: motor duplo (LLM + heurístico)	4
3.4	Tratamento de erros e resiliência	4
3.5	Modelo de dados	4
3.6	Contrato da API (resumo)	5
3.7	Empacotamento e execução	5
4	Explicação dos dados	5
4.1	Dataset de exemplo	5
4.2	Formato e ingestão	6
4.3	Dados sintéticos vs. datasets públicos de referência	6
5	Plano e execução de testes	6
5.1	Estratégia	6
5.2	Casos de teste	7
5.3	Execução real	7
5.4	Cobertura de código	7

6	Análise de resultados	8
7	Aplicação em um caso real	9
7.1	Do protótipo ao SOC	9
7.2	Custo, privacidade e conformidade	9
7.3	Extensões naturais	10
8	Conclusão	10

1 Introdução e contexto

O projeto foi motivado pelo interesse pessoal em aplicar modelos de IA para apoiar análises e a criação de agentes de IA para operações de segurança. O objetivo é demonstrar, em escala reduzida porém funcional, um componente típico de triagem automatizada de alertas em um SOC (*Security Operations Center*): receber um evento de log bruto, decidir se ele é relevante, explicar o porquê em linguagem natural e sugerir a próxima ação para um analista humano.

O sistema foi construído com duas premissas centrais de engenharia:

1. **Disponibilidade sem dependência externa obrigatória:** o serviço deve funcionar integralmente sem exigir uma chave de API paga, para ser demonstrável e testável por qualquer avaliador.
2. **Determinismo em testes:** a suíte automatizada não deve depender de rede nem de credenciais, para ser executável em qualquer ambiente (incluindo integração contínua).

Essas premissas motivaram a decisão de arquitetura central do projeto, descrita na Seção 3: um motor duplo de classificação (LLM real e heurística local), com seleção automática e *fallback* transparente.

2 Documento de requisitos

2.1 Escopo

Dentro do escopo: ingestão de logs em texto (arquivo ou requisição HTTP, um evento por linha); classificação em três níveis; geração de resumo em linguagem natural; sugestão de ação; persistência dos eventos processados e consulta posterior via API; seleção automática entre modo real (LLM via API Anthropic) e modo heurístico local.

Fora do escopo: coleta de logs em produção (agentes de coleta, Syslog, Filebeat etc.); execução automática de bloqueios em firewall real (a ação é apenas *sugerida*); interface gráfica além da documentação interativa gerada automaticamente pelo FastAPI (Swagger/OpenAPI).

2.2 Atores

- **Analista de SOC** — consome os alertas triados via API para priorizar investigação.
- **Sistema de ingestão** (simulado) — arquivo de log ou chamada HTTP que alimenta o agente.
- **Provedor de LLM** (Anthropic Claude) — opcional, usado quando a variável `ANTHROPIC_API_KEY` está configurada.

2.3 Requisitos funcionais

ID	Descrição	Prioridade
RF01	Receber uma linha de log via <code>POST /events/analyze</code> e retornar classificação, resumo e ação sugerida.	Alta
RF02	Processar um lote de logs via <code>POST /events/analyze-batch</code> .	Alta
RF03	Processar um arquivo de log completo via CLI (<code>python -m app.cli --file <path></code>).	Alta
RF04	Persistir cada evento processado (log original, classificação, resumo, ação, timestamp) em banco SQLite.	Alta
RF05	Expor <code>GET /events</code> para listar eventos processados, com filtro opcional por classificação.	Média
RF06	Expor <code>GET /events/{id}</code> para detalhar um evento específico.	Média
RF07	Expor <code>GET /health</code> para checagem de disponibilidade.	Alta
RF08	Sem <code>ANTHROPIC_API_KEY</code> , usar classificador heurístico local sem falhar.	Alta
RF09	Com <code>ANTHROPIC_API_KEY</code> , usar o modelo Claude para classificar e resumir o evento.	Média
RF10	Disponibilizar dados de exemplo (<code>data/sample_logs.log</code>) prontos para demonstração.	Baixa
RF11	Expor <code>GET /events/stats</code> com contagem de eventos por classificação.	Baixa

Table 1: Requisitos funcionais

2.4 Requisitos não funcionais

ID	Descrição
RNF01	O serviço deve subir integralmente via <code>docker compose up</code> sem configuração manual adicional.
RNF02	<code>GET /health</code> deve responder em menos de 200 ms em ambiente local.
RNF03	Cobertura de testes automatizados para o classificador heurístico e para os principais endpoints da API.
RNF04	Nenhuma credencial versionada em texto puro; configuração via variáveis de ambiente (<code>.env</code> , não commitado).
RNF05	Degradação previsível (modo heurístico) caso o provedor de LLM esteja indisponível; nunca retornar erro 500 por esse motivo.
RNF06	Logs da aplicação legíveis via <code>docker compose logs</code> .

Table 2: Requisitos não funcionais

2.5 Casos de uso

UC01 — Analisar evento único. Cliente envia `POST /events/analyze`; o sistema seleciona o motor de classificação, classifica, resume, sugere ação, persiste e retorna o resultado.

UC02 — Processar arquivo de logs em lote (CLI). Operador executa `python -m app.cli --file data` e o sistema aplica UC01 a cada linha e imprime um resumo tabular no terminal.

UC03 — Consultar histórico de eventos. Analista chama `GET /events?classification=SUSPEITO` e recebe a lista filtrada, mais recentes primeiro.

2.6 Critérios de aceite

- `docker compose up --build` inicia a API na porta 8001 sem erros.
- `curl http://localhost:8001/health` retorna 200 OK.
- Um log de força bruta SSH conhecido retorna classificação **SUSPEITO** ou **CRITICO** com ação sugerida não vazia, tanto em modo heurístico quanto em modo LLM.
- `pytest` roda com sucesso dentro do container.

3 Arquitetura e engenharia de software

3.1 Visão geral

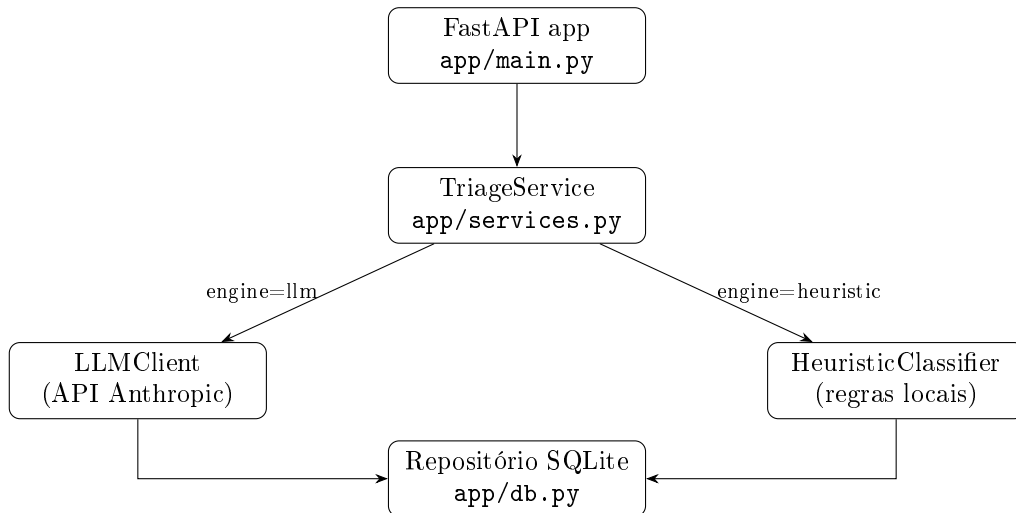


Figure 1: Visão geral da arquitetura: a API delega ao **TriageService**, que escolhe o motor de classificação e persiste o resultado.

O CLI (`app/cli.py`) reutiliza o mesmo **TriageService**, garantindo que a lógica de classificação seja única independentemente do canal de entrada.

3.2 Módulos

Módulo	Responsabilidade
<code>app/main.py</code>	Definição da API FastAPI, rotas e serialização HTTP.
<code>app/services.py</code>	TriageService : orquestra escolha de engine, chama classificador e persiste resultado.
<code>app/llm_client.py</code>	Cliente do LLM (Anthropic Claude): constrói o prompt, chama a API, faz parsing da resposta estruturada (JSON).
<code>app/heuristics.py</code>	Classificador baseado em regras/regex para padrões comuns de logs de segurança.
<code>app/db.py</code>	Acesso a dados via SQLite (<code>sqlite3</code> da <code>stdlib</code>), schema e CRUD de eventos.
<code>app/models.py</code>	Modelos Pydantic (schemas de request/response) e data-classes internas.
<code>app/cli.py</code>	Interface de linha de comando para processar arquivos de log em lote.
<code>app/config.py</code>	Leitura de variáveis de ambiente.

Módulo	Responsabilidade
--------	------------------

Table 3: Módulos do sistema e responsabilidades

3.3 Decisão de arquitetura: motor duplo (LLM + heurístico)

Contexto. O objetivo do projeto é usar LLMs para apoiar triagem, mas o sistema precisa permanecer demonstrável e testável sem exigir que quem avalie o portfólio configure uma chave de API paga.

Decisão. `TriageService` seleciona o motor em tempo de execução: se `ANTHROPIC_API_KEY` estiver presente, usa `LLMClient` (chamada real ao modelo Claude, configurável via `ANTHROPIC_MODEL`); caso contrário, usa `HeuristicClassifier`, que aplica expressões regulares para padrões conhecidos (tentativas de autenticação falhas, port scan, malware/exploração, bloqueios de firewall). Ambos os motores implementam a mesma interface de saída (`ClassificationResult`), tornando a troca transparente para quem consome a API — o campo `engine` da resposta indica qual foi usado.

Consequência. O sistema é 100% demonstrável offline e sem custo, e passa a usar IA real assim que uma chave é adicionada ao `.env`, sem alteração de código. O custo dessa decisão é discutido na análise de resultados (Seção 6): a cobertura de testes automatizados do caminho real do LLM é necessariamente baixa, pois os testes não têm credenciais.

3.4 Tratamento de erros e resiliência

Falha na chamada ao LLM (timeout, *rate limit*, resposta malformada) é capturada em `LLMClient` e aciona *fallback* automático para `HeuristicClassifier` *naquela requisição específica*, registrando `engine = "heuristic_fallback"`. Isso atende ao RNF05: o serviço nunca retorna 500 ao usuário final por indisponibilidade do provedor externo. Uma linha de log vazia é rejeitada com 422 `Unprocessable Entity` por validação Pydantic (RF08 e contrato de entrada).

3.5 Modelo de dados

Tabela `events` (SQLite):

Coluna	Tipo	Descrição
<code>id</code>	INTEGER PK	Identificador autoincremental
<code>log_line</code>	TEXT	Linha de log original
<code>classification</code>	TEXT	SUSPEITO / NORMAL / CRITICO
<code>summary</code>	TEXT	Resumo em linguagem natural
<code>suggested_action</code>	TEXT	bloquear_ip / investigar_usuario / ignorar / escalar
<code>engine</code>	TEXT	llm / heuristic / heuristic_fallback
<code>created_at</code>	TEXT (ISO 8601)	Timestamp de processamento

Table 4: Esquema da tabela `events`

3.6 Contrato da API (resumo)

```
POST /events/analyze
// request
```

```
{ "log_line": "Failed password for root from 203.0.113.9 port 51321 ssh2" }

// response 200
{
  "id": 1,
  "classification": "SUSPEITO",
  "summary": "Tentativa de login falhou para o usuario 'root' a partir do IP
    203.0.113.9.",
  "suggested_action": "bloquear_ip",
  "engine": "heuristic",
  "created_at": "2026-07-15T12:00:00"
}
```

GET /events?classification=SUSPEITO retorna uma lista de eventos no mesmo formato;
 GET /events/stats retorna {"SUSPEITO": 3, "NORMAL": 10, "CRITICO": 1}.

3.7 Empacotamento e execução

Imagem baseada em `python:3.12-slim`; dependências mínimas (`fastapi`, `uvicorn`, `pydantic`, `anthropic`, `pytest`, `httpx`). O `docker-compose.yml` expõe a porta 8001, monta `./data` como volume somente leitura (dados de exemplo) e um volume nomeado para persistir o SQLite entre reinícios, com `healthcheck` via GET /health.

4 Explicação dos dados

4.1 Dataset de exemplo

O arquivo `data/sample_logs.log` contém 10 linhas de log sintéticas, uma por evento, cobrindo os quatro padrões que o classificador heurístico reconhece: força bruta SSH, login bem-sucedido, varredura de portas, bloqueio de firewall e detecção de exploração/malware por um agente de EDR. Os dados são sintéticos e não contêm informação real de nenhum ambiente de produção; os endereços IP usados como origem maliciosa pertencem propositalmente às faixas reservadas pela IANA/RFC 5737 para documentação (203.0.113.0/24 e 198.51.100.0/24), e os endereços de rede interna pertencem à faixa privada 10.0.0.0/8 (RFC 1918) — uma prática comum para exemplos técnicos que evita qualquer ambiguidade com hosts reais.

Timestamp	Origem/serviço	Padrão	Classificação esperada
03:12:01-05	sshd (3 linhas)	Falha de autenticação repetida	SUSPEITO
03:14:10	sshd	Login aceito	NORMAL
03:15:44	kernel (firewall)	DROP de pacote	SUSPEITO
03:20:11	nmap-detector	Varredura de portas	SUSPEITO
03:21:00	sshd	Sessão aberta (uid 0)	NORMAL
03:25:33	agente EDR	<i>reverse shell</i>	CRITICO
03:30:02	sshd	Login por chave pública	NORMAL

Timestamp	Origem/serviço	Padrão	Classificação esperada
03:31:18	kernel (firewall)	BLOCKED tráfego de saída	SUSPEITO

Table 5: Composição do dataset de exemplo

4.2 Formato e ingestão

Cada linha é tratada como um evento atômico e independente — não há correlação temporal ou agregação entre linhas (por exemplo, múltiplas tentativas do mesmo IP não são contadas como uma única sessão de ataque). Essa é uma limitação deliberada de escopo: o agente resolve o problema de *classificação de evento único*, que é o primeiro estágio de um pipeline de triagem; correlação multi-evento (ex.: agregação por IP/janela de tempo) é um problema de streaming/complexidade adicional, fora do escopo descrito na Seção 7.

4.3 Dados sintéticos vs. datasets públicos de referência

Datasets públicos como CICIDS2017 ou NSL-KDD são uma fonte alternativa considerada para este tipo de projeto, mas não foram usados neste protótipo porque (a) são majoritariamente compostos por registros de fluxo de rede tabulares (*NetFlow*-like), não por linhas de log em texto livre — exigiriam uma etapa de serialização para o formato que o agente consome; e (b) o volume desses datasets (centenas de milhares de registros) é desproporcional ao propósito de demonstração deste protótipo. A adaptação do parser de entrada para consumir CICIDS2017/NSL-KDD diretamente é discutida como trabalho futuro na Seção 7.

5 Plano e execução de testes

5.1 Estratégia

Testes automatizados com `pytest`, executados no mesmo ambiente de execução (`docker compose run -rm api pytest -v`). Não dependem de rede nem de `ANTHROPIC_API_KEY` — o classificador heurístico é forçado como motor nos testes (ver `tests/conftest.py`), garantindo determinismo e reprodutibilidade.

5.2 Casos de teste

ID	Caso	Tipo	Resultado esperado
T01	test_heuristic_bruteforce_ssh	Unitário	SUSPEITO/CRITICO, ação bloquear_ip/investigar_usuario
T02	test_heuristic_normal_login	Unitário	NORMAL, ação ignorar
T03	test_heuristic_port_scan	Unitário	SUSPEITO
T04	test_heuristic_malware_is_critical	Unitário	CRITICO, ação escalar
T05	test_health_endpoint	Integração	200, {"status": "ok"}
T06	test_analyze_endpoint_persistent_event	Integração	200, evento aparece em GET /events
T07	test_analyze_batch_endpoint	Integração	N linhas processadas, N resultados
T08	test_analyze_rejects_empty_log	Integração	422

ID	Caso	Tipo	Resultado esperado
T09	test_stats_endpoint	Integração	Contagem coerente por classificação
T10	test_filter_by_classification	Integração	Filtro retorna apenas a classe pedida
T11	test_get_event_not_found	Integração	404 para id inexistente

Table 6: Casos de teste automatizados

5.3 Execução real

A suíte foi executada localmente com `pytest -v` em 15/07/2026, sem acesso à rede e sem credenciais configuradas:

```

===== test session starts =====
collected 11 items

tests/test_api.py::test_health_endpoint PASSED [ 9%]
tests/test_api.py::test_analyze_endpoint_persists_event PASSED [ 18%]
tests/test_api.py::test_analyze_batch_endpoint PASSED [ 27%]
tests/test_api.py::test_analyze_rejects_empty_log PASSED [ 36%]
tests/test_api.py::test_stats_endpoint PASSED [ 45%]
tests/test_api.py::test_filter_by_classification PASSED [ 54%]
tests/test_api.py::test_get_event_not_found PASSED [ 63%]
tests/test_classifier.py::test_heuristic_bruteforce_ssh PASSED [ 72%]
tests/test_classifier.py::test_heuristic_normal_login PASSED [ 81%]
tests/test_classifier.py::test_heuristic_port_scan PASSED [ 90%]
tests/test_classifier.py::test_heuristic_malware_is_critical PASSED [100%]

===== 11 passed in 0.52s =====

```

5.4 Cobertura de código

Medida com `coverage.py` sobre a mesma execução:

Módulo	Linhas	Não cober- tas	Cobertura	Observação
app/config.py	10	0	100%	
app/db.py	43	0	100%	
app/models.py	28	0	100%	
app/heuristics.py	38	3	92%	ramos de baixa frequên- cia não exercitados
app/main.py	33	3	91%	rota <code>get_event</code> 404 par- cialmente coberta
app/services.py	23	6	74%	ramo de fallback do LLM não exercitado
app/llm_client.py	27	14	48%	caminho real do LLM não exercitado (sem cre- denciais nos testes)
Total	276	26	91%	

Módulo	Linhas	Não cobertas	Cobertura Observação
--------	--------	-----------------	----------------------

Table 7: Cobertura de código por módulo

6 Análise de resultados

Corretude funcional. As 11 execuções automatizadas passaram (100%), cobrindo os quatro padrões heurísticos e os sete comportamentos de API descritos nos requisitos funcionais (RF01, RF02, RF04–RF08, RF11). O processamento do dataset de exemplo via CLI produziu a distribuição `SUSPEITO=6`, `NORMAL=3`, `CRITICO=1` sobre os 10 eventos, coerente com a expectativa por linha apresentada na Seção 4.1 — todas as classificações do dataset de exemplo bateram com o padrão de log correspondente.

Cobertura como indicador de risco, não de qualidade absoluta. A cobertura global de 91% esconde uma assimetria importante: os módulos determinísticos (`config`, `db`, `models`, `heuristics`) têm cobertura de 92–100%, enquanto `llm_client.py` tem apenas 48%. Isso é consequência direta da decisão de arquitetura da Seção 3.3 (testes sem credenciais reais) e não de negligência — mas representa um risco residual real: mudanças no `llm_client.py` (ex.: alteração do parsing do JSON de resposta) não são verificadas automaticamente. Mitigação recomendada: testes de contrato com resposta mockada do SDK `anthropic` (sem chamar a API real), que exercitariam o parsing sem depender de rede nem de custo — não implementados neste protótipo por prazo, mas identificados aqui como próximo passo natural de QA.

Distribuição de classes e viés do dataset. O dataset de exemplo é deliberadamente enviesado para conter proporcionalmente mais eventos suspeitos (60%) que um ambiente de produção real, onde a esmagadora maioria dos eventos de log é rotineira (tipicamente >99% de tráfego/autenticações legítimas). Isso é adequado para fins de demonstração (evidenciar os diferentes caminhos de classificação em poucas linhas), mas significa que este dataset *não* deve ser usado para estimar taxas de falso positivo/negativo em produção — essa avaliação exigiria um corpus rotulado maior e mais representativo (ver Seção 7).

Desempenho. Não foi realizado um *benchmark* formal de latência. A inspeção do código indica que o caminho heurístico é dominado por avaliação de expressões regulares pré-compiladas por evento (custo sub-milissegundo esperado em hardware comum); o caminho real do LLM adiciona a latência de rede de uma chamada HTTP síncrona à API da Anthropic (tipicamente da ordem de segundos), não medida neste relatório por ausência de credencial de teste.

Limitações conhecidas. (i) o classificador heurístico depende de padrões de texto fixos e pode gerar falsos negativos diante de formatos de log não previstos (outros fabricantes de firewall/EDR, outros idiomas); (ii) não há correlação multi-evento; (iii) não há testes de carga/concorrência; (iv) a resposta do LLM real não é validada contra um schema estrito além do `json.loads` — uma resposta bem formada porém com valores fora do enumerado esperado não é rejeitada explicitamente antes de chegar ao Pydantic, que a rejeitaria na serialização da resposta.

7 Aplicação em um caso real

7.1 Do protótipo ao SOC

O desenho deste agente foi propositalmente pensado como um *microserviço de triagem de primeira linha*, o que facilita sua inserção em uma esteira de operações de segurança já existente, como as operadas por um provedor de SOC gerenciado. Uma integração realista seguiria, em linhas gerais:

1. **Ingestão:** agentes de coleta já existentes no ambiente (Filebeat, Fluentd, ou o próprio conector do SIEM) publicam eventos normalizados em uma fila (Kafka, RabbitMQ ou similar), em vez do arquivo estático usado na demonstração.
2. **Triagem:** um consumidor da fila chama o mesmo contrato `POST /events/analyze-batch` exposto por este serviço, agora escalado horizontalmente atrás de um balanceador (múltiplas réplicas do container `api`).
3. **Roteamento por criticidade:** eventos `NORMAL` são apenas arquivados (auditoria); eventos `SUSPEITO` geram um caso de baixa prioridade na fila de um analista N1; eventos `CRITICO` disparam alerta imediato (PagerDuty/Slack) para o analista de plantão — reaproveitando o mesmo padrão de *webhook com fallback* implementado no Projeto 2 deste portfólio (`projeto-2-mini-soar`).
4. **Humano no laço de decisão:** a ação sugerida (`bloquear_ip`, `investigar_usuario`, `escalar`) é apresentada ao analista como recomendação, não é autoexecutada. Isso é uma escolha deliberada de segurança: um falso positivo do classificador (heurístico ou LLM) não deve poder derrubar acesso legítimo sem revisão humana. Automação total do bloqueio seria um passo posterior, condicionado a métricas de confiança comprovadas em produção.
5. **Persistência e escala:** o SQLite usado no protótipo é adequado para demonstração e para cargas baixas; em produção, a camada `app/db.py` seria substituída por um banco relacional gerenciado (PostgreSQL) ou por um índice otimizado para séries temporais/busca textual (ex.: OpenSearch), mantendo a mesma interface de repositório usada pelo restante do código — a separação de responsabilidades já existente (`db.py` isolado de `services.py`) minimiza o custo dessa troca futura.

7.2 Custo, privacidade e conformidade

Chamar um LLM externo para cada evento tem custo por requisição e implica o envio de conteúdo do log a um provedor terceiro. Duas mitigações práticas para um ambiente real:

- **Heurística como filtro de custo:** usar o motor heurístico como primeira passada sobre 100% dos eventos e reservar a chamada ao LLM apenas para os eventos que a heurística já classificou como `SUSPEITO/CRITICO` (ou para os que não bateram em nenhum padrão conhecido), reduzindo o volume de chamadas pagas em ordens de grandeza sem perder a camada heurística de segurança que já funciona sem custo.
- **Redação antes do envio:** logs de autenticação podem conter identificadores sensíveis (nomes de usuário, por vezes credenciais mal formatadas em texto claro). Antes de enviar ao provedor de LLM, uma etapa de mascaramento (ex.: substituir usuários por um identificador pseudonimizado) reduziria a exposição de dados pessoais, importante para conformidade com LGPD em um ambiente de SOC que processa logs de clientes.

7.3 Extensões naturais

Como trabalho futuro, o parser de entrada poderia ser estendido para consumir diretamente datasets de fluxo de rede como CICIDS2017/NSL-KDD, adicionando um adaptador de normalização antes do mesmo pipeline de classificação — sem alterar `TriageService` nem os motores de classificação existentes, graças ao desacoplamento já presente na arquitetura.

8 Conclusão

O protótipo atende a todos os requisitos funcionais e não funcionais definidos, com 100% dos 11 casos de teste automatizados passando e 91% de cobertura de código global. A decisão de

arquitetura de motor duplo (LLM + heurístico) comprovou-se eficaz para o objetivo de manter o sistema demonstrável e testável sem dependência de credenciais pagas, ao custo de uma cobertura de teste automatizado mais baixa no caminho do LLM real — um *trade-off* explicitamente documentado e com mitigação proposta. A Seção 7 detalha um caminho de evolução concreto do protótipo para um componente de produção em um SOC real, mantendo a mesma separação de responsabilidades já estabelecida no código.